



1 Introdução

Pretende-se desenvolver um sistema de comunicação em tempo real para os funcionários de uma empresa poderem trocar rapidamente informações entre si.

De forma diferente dos normais programas de “chat”, este sistema não funciona baseado no conceito de “mensagem” (usando um botão “send” para enviar uma mensagem previamente escrita), mas sim num sistema letra-a-letra (cada “keypress” é enviado imediatamente). Assim, os utilizadores recebem as mensagens ao mesmo tempo que estas estão a ser escritas!

O facto de o sistema ter de funcionar em tempo real, faz-nos optar por uma implementação baseada no protocolo UDP, mas é necessário definir um protocolo de aplicação, para que a troca de dados ocorra sem problemas.

1.1 Equipas de trabalho

- O trabalho deve ser executado em equipas de 4 alunos, sendo sugerido que trabalhem em conjunto na definição do protocolo e depois se dividam em duas equipas, em que uma implementa o cliente e outra implementa o servidor.
- As equipas devem ser registadas no moodle, sendo-lhes posteriormente atribuído um número único de identificação.

2 Trabalho Base (8 valores)

2.1 Protocolo

O protocolo deve especificar como se processa a interacção entre servidor e clientes, descrevendo todos os tipo de pedidos, respostas e outras acções a ser executadas pelos clientes e pelo servidor, de forma a que a comunicação possa acontecer sem problemas.

Também deve especificar as acções apropriadas da parte do servidor para quando acontecem coisas não especificadas no protocolo (e.g., quando aparecem “rogue clients” ou clientes mal implementados, etc.)¹

2.2 Servidor e Cliente

O servidor aceita datagramas UDP na porta 12345.

Cada cliente envia mensagens ao servidor, podendo estas ser de vários tipos:

- Autenticação (extra 3.1);
- Avisos (se implementados);

¹Para inspiração, pode ser usada a RFC 1459 [<https://www.rfc-editor.org/rfc/rfc1459.html>] que define o protocolo IRC.

- Caracteres (letras) das mensagens;
- Outras...

Na maioria dos casos, não deverá haver lugar a respostas do servidor às mensagens dos clientes, excepto quando estritamente necessário (e.g., quando o cliente não se apercebe que não se autenticou devidamente, quando o cliente pede especificamente uma resposta, etc.).

Na implementação base, têm de estar implementadas as seguintes funcionalidades:

1. O servidor deve conseguir atender múltiplos clientes em simultâneo.
2. Os clientes podem enviar mensagens (texto) ao servidor, que as reenvia para todos os outros clientes.
3. Os clientes podem receber mensagens do servidor, que podem corresponder a mensagens (texto) de outros clientes ou outro tipo de mensagens do servidor.
4. O servidor pode enviar mensagens aos clientes a indicar que um novo cliente “apareceu” ou “abandonou” o serviço.
5. O cliente deve enviar uma mensagem de “despedida” quando abandona o serviço.
6. O servidor deve detectar quando um cliente “abandonou” o serviço sem enviar a mensagem de despedida.

3 Extras

3.1 Autenticação (3 valores)

Deverá ser criado um sistema básico de autenticação através de **username** e **password**, de forma a que os clientes passem a ser identificados inequivocamente.

Assim, as mensagens podem ser direccionadas a utilizadores específicos, através do seu username, bem como serem marcadas com o username de quem as enviou. Isto implica que os clientes passem a poder indicar para quem é a mensagem (se para um utilizador, se para todos) e que o servidor as trate de forma adequada, enviando para o(s) destinatário(s) correcto(s).

Será necessário guardar os usernames e passwords numa estrutura persistente (e.g., num ficheiro), para que não sejam perdidos quando o servidor reiniciar.

3.2 Controlo de erros (3 valores)

Como estamos a usar UDP, é possível que algumas mensagens se percam na rede. Assim sendo, deve haver uma forma de, periodicamente, ser verificado o número de mensagens enviadas desde o último “checkpoint”, podendo assim corrigir-se algum erro que tenha ocorrido entretanto.

3.3 Transferência de ficheiros (4 valores)

Deverá ser possível a um utilizador do sistema iniciar um processo de transferência de um ficheiro para um ou mais utilizadores. Este processo funciona da seguinte forma:

1. O cliente avisa os destinatários que vai enviar um ficheiro;
2. O cliente abre um servidor TCP numa porta;
3. Os destinatários que aceitem receber o ficheiro ligam (`connect()`) a essa porta e recebem o ficheiro;
4. Passado algum tempo (definido no protocolo), o processo termina e o ficheiro deixa de estar disponível.

3.4 Grupos (3 valores)

À semelhança das aplicações actuais de chat, deve ser possível aos utilizadores criarem grupos onde adicionam outros utilizadores, de forma a poderem enviar mensagens para o grupo (todos os utilizadores recebem as mensagens do grupo).

Os grupos devem ter um nome (funciona como `username`) e devem persistir se o servidor for reiniciado. Deve também ser possível remover elementos do grupo e apagar grupos (só o criador do grupo o pode apagar).

4 Problemas, Escolhas e Omissões no enunciado

É normal que haja problemas ao desenvolver o protocolo para este sistema. Sempre que não haja uma recomendação “oficial” de como resolver um problema, ficará a cargo da equipa propor uma solução de compromisso para esse problema, sendo esta solução objecto da avaliação final.

Um exemplo será o tratamento de mensagens simultâneas de vários utilizadores e o tratamento daquelas pelo cliente (não dá muito jeito mostrar letras de várias mensagens de vários clientes “misturadas” no output...). Devemos “bloquear” os clientes quando outro está a “falar”, ou usamos a técnica dos “walkie-talkies”, em que quem termina de falar diz “câmbio”?

Cabe a cada equipa resolver este tipo de situações de forma eficiente e eficaz.

5 Entregas

O trabalho é avaliado através de uma entrega final. No entanto, haverá duas “entregas” intermédias (“milestones”), a realizar nas aulas práticas, onde as equipas de trabalho devem mostrar o seu progresso, podendo também esclarecer dúvidas e receber indicações úteis sobre como prosseguir. Embora não contem para a avaliação final, a falta nas apresentações das *milestones* será penalizada com um desconto de 20% (por cada falta) na nota final do trabalho.

A entrega final será objecto de apresentação/discussão e cada elemento da equipa será avaliado pela sua prestação individual.

5.1 Milestone 1: Protocolo

Nesta fase, as equipas devem apresentar o protocolo que desenvolveram para a sua aplicação. Será útil que o protocolo contemple todas as partes (base + extras), mesmo que estas não cheguem a ser implementadas.

Deadline: **semana de 7 a 11 de abril.**

5.2 Milestone 2: Trabalho Base

Nesta fase, as equipas devem apresentar um protótipo funcional que implemente a parte base do trabalho (cliente + servidor).

Deadline: **semana de 5 a 9 de maio.**

5.3 Entrega Final: Trabalho Base + Extras

Aqui, as equipas devem entregar o trabalho finalizado, acompanhado por um relatório de implementação onde é apresentado o protocolo, as escolhas de implementação e uma avaliação crítica e objectiva do funcionamento da aplicação.

Deadline: **9 de junho.**

5.3.1 Formato da entrega final

A entrega final deverá ser feita no moodle, no espaço próprio para o efeito, através de um ficheiro chamado **equipaEE.tar.gz** ou **equipaEE.zip**, onde EE é o número que foi atribuído à equipa. Ao descompactar, o trabalho deve gerar uma directoria **equipaEE/** que por sua vez contem 3 directorias:

```
equipaEE/
├─ client      # contém o código do cliente
├─ report      # contém o relatório em formato .txt ou .pdf (nunca .doc ou .docx!)
└─ server      # contém o código do servidor
```

Bom trabalho!